

Gestión de Aplicaciones Web Empresariales

Práctica en Clase



Proyecto ASP.NET Core MVC

Implementación de Modelo, Vista, Controlador
con Validaciones Server-Side y AntiForgeryToken

Estudiante:

Justyn Keith Cruz Perez

Fecha: 18 de junio de 2026

Proyecto: MiPrimerMVC

Tecnología: ASP.NET Core 8.0 MVC

SDK: .NET 8.0.422

Práctica individual — Entrega correspondiente al período académico 2026

ASP.NET Core MVC — Validaciones Server-Side — Data Annotations —
AntiForgeryToken

Índice

1. Introducción	2
2. Paso 1: Creación del Proyecto	2
2.1. Comando de inicialización	2
2.2. Salida del comando	3
3. Paso 2: Modelo Producto con Data Annotations	3
3.1. ¿Qué son las Data Annotations?	3
3.2. Código del Modelo	3
3.3. Descripción de los Atributos de Validación	5
4. Paso 3: ProductoController con Acciones Index y Create	5
4.1. Estructura del Controlador	5
4.2. Análisis de las Acciones	6
4.2.1. Acción Index (GET)	6
4.2.2. Acción Create GET	7
4.2.3. Acción Create POST con Validación	7
5. Paso 4: Vistas Razor con AntiForgeryToken	7
5.1. Vista Index.cshtml	8
5.2. Vista Create.cshtml con AntiForgeryToken	9
5.3. Elementos de Validación en la Vista	11
6. Paso 5: Verificación de la Validación Server-Side	11
6.1. Ejecución de la Aplicación	11
6.2. Prueba 1: Validación con Formulario Vacío	12
6.2.1. Resultado: Errores de validación en pantalla	12
6.3. Prueba 2: Registro Exitoso de un Producto	13
6.3.1. Resultado: Producto agregado exitosamente	13
7. Análisis del Patrón MVC en el Proyecto	13
7.1. Flujo de Solicitudes	13
7.2. Separación de Responsabilidades	14
8. Conclusiones	14
9. Referencias	15

1. Introducción

ASP.NET Core MVC es un framework de desarrollo web de código abierto desarrollado por Microsoft que implementa el patrón arquitectónico **Modelo-Vista-Controlador (MVC)**. Este patrón separa claramente las responsabilidades de una aplicación web en tres componentes interdependientes:

- **Modelo (Model):** Representa los datos y la lógica de negocio de la aplicación. Define la estructura de los datos y las reglas de validación mediante *Data Annotations*.
- **Vista (View):** Capa de presentación que genera la interfaz de usuario. En ASP.NET Core MVC se utiliza el motor de plantillas **Razor** (.cshtml).
- **Controlador (Controller):** Gestiona las solicitudes HTTP, coordina la comunicación entre Modelo y Vista, y aplica la lógica de la aplicación.

✓ Objetivo

Crear un proyecto ASP.NET Core MVC funcional que incluya:

1. Creación del proyecto con `dotnet new mvc`.
2. Modelo `Producto` con validaciones mediante *Data Annotations*.
3. Controlador `ProductoController` con acciones `Index` y `Create`.
4. Vistas Razor con `AntiForgeryToken`.
5. Verificación de validaciones a nivel servidor (server-side).

2. Paso 1: Creación del Proyecto

2.1. Comando de inicialización

El primer paso consiste en crear la estructura base del proyecto MVC mediante la CLI de .NET. Se ejecutó el siguiente comando:

```
dotnet new mvc -n MiPrimerMVC
```

Listing 1: Creación del proyecto MiPrimerMVC

Este comando genera automáticamente la estructura completa de un proyecto ASP.NET Core MVC, incluyendo:

Cuadro 1: Estructura de archivos generada por `dotnet new mvc`

Directorio / Archivo	Descripción
Controllers/	Carpeta para los controladores
Models/	Carpeta para los modelos de datos
Views/	Carpeta para las vistas Razor (.cshtml)
wwwroot/	Archivos estáticos (CSS, JS, imágenes)
Program.cs	Punto de entrada y configuración de servicios
appsettings.json	Configuración de la aplicación
MiPrimerMVC.csproj	Archivo de proyecto .NET

2.2. Salida del comando

```
La plantilla "Aplicacion web de ASP.NET Core (Modelo-Vista-
Controlador)"
se creo correctamente.

Procesando acciones posteriores a la creacion...
Restaurando MiPrimerMVC\MiPrimerMVC.csproj:
  Se ha restaurado MiPrimerMVC.csproj (en 187 ms).
Restauracion realizada correctamente.
```

Listing 2: Resultado de la creación del proyecto

Concepto Clave

El comando `dotnet new mvc` genera una plantilla base con soporte para Bootstrap 5, jQuery Validation, y el motor Razor, listos para usar sin configuración adicional.

3. Paso 2: Modelo Producto con Data Annotations

3.1. ¿Qué son las Data Annotations?

Las **Data Annotations** son atributos del espacio de nombres `System.ComponentModel.DataAnnotations` que permiten definir reglas de validación directamente en el modelo. ASP.NET Core MVC las interpreta para:

- Validar datos en el servidor antes de procesarlos (**server-side validation**).
- Generar atributos HTML para validación en el cliente.
- Configurar la visualización de campos en las vistas mediante Tag Helpers.

3.2. Código del Modelo

Se creó el archivo `Models/Producto.cs` con las siguientes propiedades y anotaciones:

```
1 using System.ComponentModel.DataAnnotations;
2
3 namespace MiPrimerMVC.Models;
4
5 public class Producto
6 {
7     [Key]
8     public int Id { get; set; }
9
10    [Required(ErrorMessage = "El nombre del producto es
11        obligatorio.")]
12    [StringLength(100, MinimumLength = 3,
13        ErrorMessage = "El nombre debe tener entre 3 y 100
14        caracteres.")]
15    [Display(Name = "Nombre del Producto")]
16    public string Nombre { get; set; } = string.Empty;
17
18    [Required(ErrorMessage = "El precio es obligatorio.")]
19    [Range(0.01, 10000.00,
20        ErrorMessage = "El precio debe estar entre 0.01 y
21        10000.00.")]
22    [DataType(DataType.Currency)]
23    [Display(Name = "Precio")]
24    public decimal Precio { get; set; }
25
26    [StringLength(250,
27        ErrorMessage = "La descripcion no puede superar los 250
28        caracteres.")]
29    [Display(Name = "Descripcion")]
30    public string? Descripcion { get; set; }
31
32    [Required(ErrorMessage = "El stock es obligatorio.")]
33    [Range(0, 1000,
34        ErrorMessage = "El stock debe ser un valor no negativo
35        (maximo 1000).")]
36    [Display(Name = "Cantidad en Stock")]
37    public int Stock { get; set; }
38 }
```

Listing 3: Models/Producto.cs — Modelo con Data Annotations

3.3. Descripción de los Atributos de Validación

Cuadro 2: Atributos de validación utilizados en el modelo Producto

Atributo	Descripción
[Key]	Marca la propiedad como clave primaria del modelo.
[Required]	El campo es obligatorio; no puede ser nulo ni vacío.
[StringLength]	Limita la longitud del texto (máximo y mínimo).
[Range]	Valida que el valor esté dentro del rango especificado.
[DataType]	Define el tipo de datos para formateo (ej. Currency).
[Display]	Establece el nombre de etiqueta mostrado en la vista.

4. Paso 3: ProductoController con Acciones Index y Create

4.1. Estructura del Controlador

Se creó el archivo `Controllers/ProductoController.cs`. En ASP.NET Core MVC, todo controlador debe heredar de la clase base `Controller` y sus métodos públicos corresponden a **acciones** que responden a solicitudes HTTP.

```

1 using Microsoft.AspNetCore.Mvc;
2 using MiPrimerMVC.Models;
3
4 namespace MiPrimerMVC.Controllers;
5
6 public class ProductoController : Controller
7 {
8     // Lista estatica en memoria para simular una base de datos
9     private static readonly List<Producto> _productos = new
10         List<Producto>
11     {
12         new Producto { Id = 1, Nombre = "Laptop Dell Inspiron",
13             Precio = 749.99m,
14             Descripcion = "Intel i5, 8GB RAM, 256GB SSD", Stock
15                 = 15 },
16         new Producto { Id = 2, Nombre = "Mouse Gamer RGB",
17             Precio = 29.99m,
18             Descripcion = "Mouse ergonomico con luces led",
19             Stock = 50 },
20         new Producto { Id = 3, Nombre = "Teclado Mecanico
21             Redragon",

```

```
18         Precio = 59.99m,  
19         Descripcion = "Teclado layout espanol, switches  
           azules", Stock = 30 }  
20     };  
21  
22     // GET: /Producto/Index  
23     public IActionResult Index()  
24     {  
25         return View(_productos);  
26     }  
27  
28     // GET: /Producto/Create  
29     public IActionResult Create()  
30     {  
31         return View(new Producto());  
32     }  
33  
34     // POST: /Producto/Create  
35     [HttpPost]  
36     [ValidateAntiForgeryToken]  
37     public IActionResult Create(Producto producto)  
38     {  
39         if (ModelState.IsValid)  
40         {  
41             // Asignar un ID auto-incremental simple  
42             producto.Id = _productos.Any()  
43                 ? _productos.Max(p => p.Id) + 1 : 1;  
44             _productos.Add(producto);  
45  
46             // Redireccionar al listado de productos  
47             return RedirectToAction(nameof(Index));  
48         }  
49  
50         // Si el modelo no es valido, regresar la vista con  
51         // errores  
52         return View(producto);  
53     }  
}
```

Listing 4: Controllers/ProductoController.cs

4.2. Análisis de las Acciones

4.2.1. Acción Index (GET)

```
1 public IActionResult Index()  
2 {  
3     return View(_productos);  
4 }
```

Listing 5: Acción Index — Muestra la lista de productos

Responde a solicitudes GET /Producto/Index. Pasa la lista estática `_productos` a la vista `Views/Producto/Index.cshtml` como modelo fuertemente tipado.

4.2.2. Acción Create GET

```
1 public IActionResult Create()
2 {
3     return View(new Producto());
4 }
```

Listing 6: Acción Create GET — Renderiza el formulario vacío

Responde a solicitudes GET /Producto/Create. Envía un objeto `Producto` vacío a la vista para que se inicialicen correctamente los campos del formulario.

4.2.3. Acción Create POST con Validación

```
1 [HttpPost]
2 [ValidateAntiForgeryToken]
3 public IActionResult Create(Producto producto)
4 {
5     if (ModelState.IsValid)
6     {
7         producto.Id = _productos.Any()
8             ? _productos.Max(p => p.Id) + 1 : 1;
9         _productos.Add(producto);
10        return RedirectToAction(nameof(Index));
11    }
12
13    return View(producto); // Regresa con los errores visibles
14 }
```

Listing 7: Acción Create POST — Validación server-side

Conceptos Clave del POST

- **[HttpPost]**: Restringe la acción para que solo responda a solicitudes HTTP POST.
- **[ValidateAntiForgeryToken]**: Verifica el token CSRF incluido en el formulario, protegiendo contra ataques de Cross-Site Request Forgery.
- **ModelState.IsValid**: Propiedad que indica si todos los datos recibidos cumplen las reglas de validación del modelo (*Data Annotations*).
- Cuando **ModelState.IsValid** es **false**, se retorna la misma vista con el objeto `producto` para que los mensajes de error sean visibles al usuario.

5. Paso 4: Vistas Razor con AntiForgeryToken

5.1. Vista Index.cshtml

La vista del listado presenta todos los productos en una tabla responsiva con estilos de Bootstrap 5.

```

1 @model IEnumerable<MiPrimerMVC.Models.Producto>
2
3 @{
4     ViewData["Title"] = "Listado de Productos";
5 }
6
7 <div class="container mt-4">
8     <div class="d-flex justify-content-between
9         align-items-center mb-4">
10         <h1 class="display-5 fw-bold text-primary">
11             Catalogo de Productos
12         </h1>
13         <a asp-action="Create" class="btn btn-success btn-lg">
14             Nuevo Producto
15         </a>
16     </div>
17
18     <div class="card shadow-sm border-0 rounded-3">
19         <table class="table table-hover table-striped mb-0">
20             <thead class="table-dark">
21                 <tr>
22                     <th>ID</th>
23                     <th>Nombre</th>
24                     <th>Descripcion</th>
25                     <th>Precio</th>
26                     <th>Stock</th>
27                 </tr>
28             </thead>
29             <tbody>
30                 @foreach (var item in Model)
31                 {
32                     <tr>
33                         <td>@item.Id</td>
34                         <td>@item.Nombre</td>
35                         <td>@item.Descripcion</td>
36                         <td>@item.Precio.ToString("C")</td>
37                         <td>@item.Stock</td>
38                     </tr>
39                 }
40             </tbody>
41         </table>
42     </div>

```

Listing 8: Views/Producto/Index.cshtml — Vista de listado

5.2. Vista Create.cshtml con AntiForgeryToken

```

1 @model MiPrimerMVC.Models.Producto
2
3 @{
4     ViewData["Title"] = "Nuevo Producto";
5 }
6
7 <div class="container mt-4">
8     <div class="row justify-content-center">
9         <div class="col-md-8 col-lg-6">
10             <h1 class="h3 fw-bold text-primary">
11                 Registrar Nuevo Producto
12             </h1>
13
14             <div class="card shadow-sm border-0 rounded-3">
15                 <div class="card-body p-4">
16                     <form asp-action="Create" method="post">
17
18                         <!-- AntiForgery Token de seguridad
19                          CSRF -->
20                         @Html.AntiForgeryToken()
21
22                         <!-- Resumen de errores del modelo -->
23                         <div asp-validation-summary="ModelOnly"
24                             class="text-danger mb-3">
25
26                         <!-- Campo Nombre -->
27                         <div class="mb-3">
28                             <label asp-for="Nombre"
29                                 class="form-label fw-semibold">
30
31                             <input asp-for="Nombre"
32                                 class="form-control" />
33                             <span asp-validation-for="Nombre"
34                                 class="text-danger small">
35
36                             </span>
37                         </div>
38
39                         <!-- Campo Descripcion -->
40                         <div class="mb-3">
41                             <label asp-for="Descripcion"
42                                 class="form-label">
43
44                             </label>
45                             <textarea asp-for="Descripcion"
46                                 class="form-control"
47                                 rows="3">
48
49                             </textarea>
50                             <span
51                                 asp-validation-for="Descripcion"

```

```

44         class="text-danger small">
45     </span>
46 </div>
47
48 <!-- Precio y Stock en fila -->
49 <div class="row">
50     <div class="col-md-6 mb-3">
51         <label asp-for="Precio"
52             class="form-label">
53         </label>
54         <input asp-for="Precio"
55             class="form-control"
56             type="number"
57             step="0.01" />
58         <span
59             asp-validation-for="Precio"
60             class="text-danger small">
61         </span>
62     </div>
63     <div class="col-md-6 mb-3">
64         <label asp-for="Stock"
65             class="form-label">
66         </label>
67         <input asp-for="Stock"
68             class="form-control"
69             type="number" />
70         <span asp-validation-for="Stock"
71             class="text-danger small">
72         </span>
73     </div>
74 </div>
75
76 <button type="submit" class="btn
77     btn-primary btn-lg">
78     Guardar Producto
79 </button>
80 </form>
81 </div>
82 </div>
83 </div>
84
85 @section Scripts {
86     @{await
87         Html.RenderPartialAsync("_ValidationScriptsPartial");}
88 }

```

Listing 9: Views/Producto/Create.cshtml — Formulario con AntiForgeryToken

5.3. Elementos de Validación en la Vista

Cuadro 3: Tag Helpers de validación en las vistas Razor

Tag Helper / HTML Helper	Función
@Html.AntiForgeryToken()	Genera un campo oculto con el token CSRF que debe enviarse con cada solicitud POST.
asp-validation-summary	Muestra un resumen de todos los errores del modelo en un solo lugar.
asp-validation-for	Muestra el mensaje de error específico para un campo particular del modelo.
asp-for	Enlaza el campo del formulario a la propiedad del modelo (genera <code>name</code> , <code>id</code> y <code>value</code> automáticamente).
asp-action	Especifica la acción del controlador a la que se enviará el formulario.

Importante

¿Por qué es importante el AntiForgeryToken?

El *Cross-Site Request Forgery* (CSRF) es un ataque en el que un sitio malicioso envía solicitudes falsas a tu aplicación en nombre de un usuario autenticado. El `AntiForgeryToken` genera un token único por sesión que se verifica en cada solicitud POST. Si el token no está presente o no coincide, ASP.NET Core rechaza la petición con un error 400.

6. Paso 5: Verificación de la Validación Server-Side

6.1. Ejecución de la Aplicación

La aplicación se ejecutó con el siguiente comando:

```
dotnet run --launch-profile "http"
```

Listing 10: Comando para iniciar la aplicación

```
Compilando...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5087
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
```

Listing 11: Salida del servidor al iniciar

6.2. Prueba 1: Validación con Formulario Vacío

Se navegó a `http://localhost:5087/Producto/Create` y se hizo clic en el botón “**Guardar Producto**” sin completar ningún campo del formulario.

Flujo de validación server-side

1. El navegador envía una solicitud `POST` al servidor con los campos del formulario vacíos.
2. El *Model Binder* de ASP.NET Core vincula los datos recibidos al objeto `Producto`.
3. ASP.NET Core evalúa las *Data Annotations* del modelo y registra los errores en `ModelState`.
4. La condición `ModelState.IsValid` resulta `false`.
5. El controlador devuelve la misma vista con los mensajes de error para cada campo.

6.2.1. Resultado: Errores de validación en pantalla

Al enviar el formulario vacío, el servidor respondió con los mensajes de error definidos en las *Data Annotations* del modelo, tal como se observa en la captura siguiente:



Figura 1: Validación server-side: mensajes de error al enviar el formulario vacío

Los mensajes de error mostrados fueron:

- “*El nombre del producto es obligatorio.*” — Producido por `[Required]` en la propiedad `Nombre`.
- “*El precio debe estar entre 0.01 y 10000.00.*” — Producido por `[Range]` en la propiedad `Precio` (el valor por defecto 0 queda fuera del rango mínimo).

6.3. Prueba 2: Registro Exitoso de un Producto

A continuación, se completó el formulario con los siguientes datos válidos:

Cuadro 4: Datos utilizados para el registro del producto de prueba

Campo	Valor ingresado
Nombre del Producto	Impresora HP LaserJet
Descripción	Impresora laser de alta velocidad, Blanco y Negro
Precio	\$189.99
Cantidad en Stock	15

Al hacer clic en “**Guardar Producto**”, la acción `Create POST` verificó que `ModelState.IsValid` era `true`, añadió el producto a la lista en memoria y ejecutó `RedirectToAction(nameof(Index))`, redirigiendo al usuario a la vista de listado.

6.3.1. Resultado: Producto agregado exitosamente



MiPrimerMVC Home Privacy				
Catálogo de Productos				
Gestión de inventario y precios de productos.				
Nuevo Producto				
ID	Nombre	Descripción	Precio	Stock
1	Laptop Dell Inspiron	Intel i5, 8GB RAM, 256GB SSD	\$749,99	15 unids.
2	Mouse Gamer RGB	Mouse ergonómico con luces led	\$29,99	50 unids.
3	Teclado Mecánico Redragon	Teclado layout español, switches azules	\$59,99	30 unids.
4	Impresora HP LaserJet	Impresora laser de alta velocidad, Blanco y Negro	\$189,99	Sin stock
5	Impresora HP LaserJet	Impresora laser de alta velocidad, Blanco y Negro	\$189,99	15 unids.
© 2026 - MiPrimerMVC - Privacy				

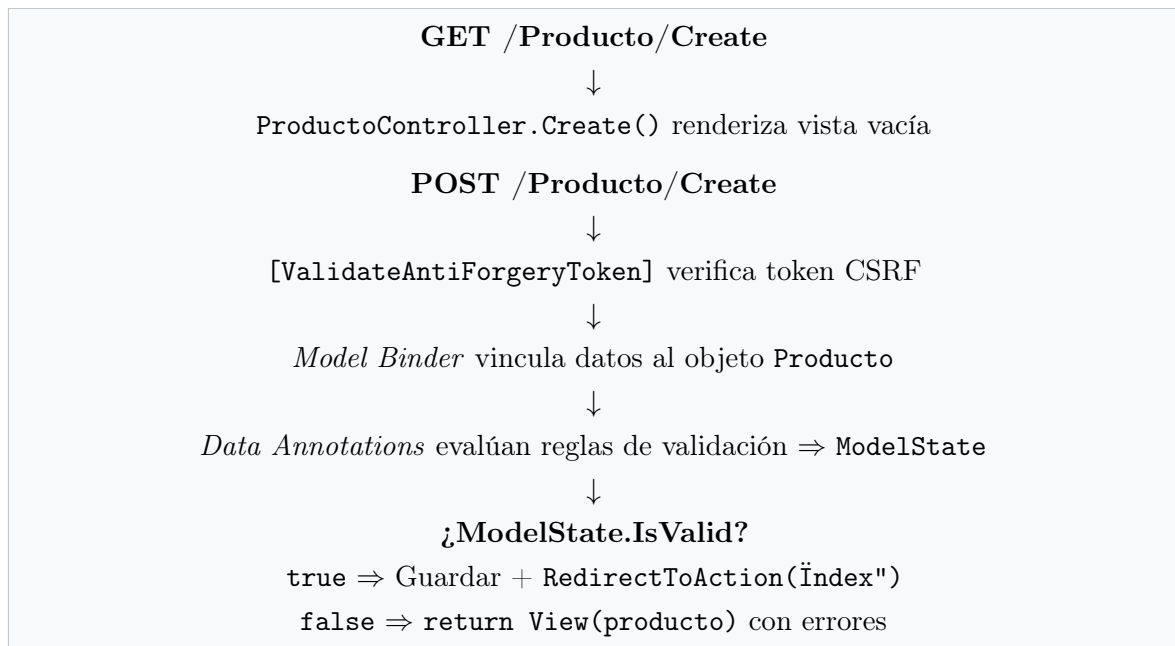
Figura 2: Vista Index mostrando el producto “Impresora HP LaserJet” agregado correctamente

El producto con ID 5 (“Impresora HP LaserJet”, \$189.99, 15 unids.) quedó registrado y visible en el catálogo, confirmando el correcto funcionamiento del flujo MVC completo: Formulario → Controlador → Modelo → Redirección → Vista.

7. Análisis del Patrón MVC en el Proyecto

7.1. Flujo de Solicitudes

El siguiente diagrama describe el flujo completo de una solicitud en la aplicación:



7.2. Separación de Responsabilidades

Cuadro 5: Responsabilidades de cada capa MVC en el proyecto

Capa	Archivo	Responsabilidad
Modelo	Models/Producto.cs	Datos + reglas de negocio
Vista	Views/Producto/Index.cshtml	Presentación del listado
Vista	Views/Producto/Create.cshtml	Formulario de registro
Controlador	Controllers/ProductoController.cs	Lógica de solicitudes HTTP
Enrutamiento	Program.cs	Configuración de rutas

8. Conclusiones

El desarrollo de esta práctica permitió comprender en profundidad los siguientes conceptos fundamentales del desarrollo web con ASP.NET Core MVC:

1. **Patrón MVC:** La separación de responsabilidades entre Modelo, Vista y Controlador facilita el mantenimiento y la escalabilidad del código. Cada capa tiene una función específica y bien delimitada.
2. **Data Annotations:** Los atributos de validación como `[Required]`, `[Range]` y `[StringLength]` permiten definir reglas de negocio directamente en el modelo, sin necesidad de código adicional en el controlador. ASP.NET Core los procesa automáticamente.
3. **Validación Server-Side:** La verificación de `ModelState.IsValid` en el controlador garantiza que los datos sean válidos antes de ser procesados, independientemente de si el cliente tiene JavaScript habilitado o no. Esto representa una capa de seguridad fundamental.

4. **AntiForgeryToken:** La combinación de `@Html.AntiForgeryToken()` en la vista y `[ValidateAntiForgeryToken]` en el controlador protege la aplicación contra ataques CSRF, asegurando que solo formularios generados por la propia aplicación puedan enviar solicitudes POST.
5. **Tag Helpers de Razor:** Los atributos `asp-for`, `asp-validation-for` y `asp-validation-summary` simplifican enormemente la creación de formularios enlazados al modelo, generando automáticamente el HTML necesario para la validación y el enlace de datos.

✓ Resultado Final

El proyecto **MiPrimerMVC** fue creado, implementado y verificado exitosamente. La validación server-side funciona correctamente, los mensajes de error se muestran al usuario cuando el formulario está incompleto, y el registro de nuevos productos redirige correctamente al listado. Todos los pasos de la directriz de la práctica fueron completados satisfactoriamente.

9. Referencias

1. Microsoft. (2024). *Tutorial: Get started with ASP.NET Core MVC*. <https://learn.microsoft.com/en-us/aspnet/core/tutorials/first-mvc-app/start-mvc>
2. Microsoft. (2024). *Model validation in ASP.NET Core MVC*. <https://learn.microsoft.com/en-us/aspnet/core/mvc/models/validation>
3. Microsoft. (2024). *Prevent Cross-Site Request Forgery (XSRF/CSRF) attacks in ASP.NET Core*. <https://learn.microsoft.com/en-us/aspnet/core/security/anti-request-forgery>
4. Microsoft. (2024). *Tag Helpers in forms in ASP.NET Core*. <https://learn.microsoft.com/en-us/aspnet/core/mvc/views/working-with-forms>